Technical Report

RAG Chatbot for AWS Customer Agreement Q&A

1. Overview

This project is a retrieval-augmented generation (RAG) chatbot that answers questions over a single legal document the AWS Customer Agreement with grounded, citation-backed answers and a built-in refusal path for out-of-scope questions. The stack is:

- **FastAPI** backend API
- **Streamlit** chat and analytics UI
- **ChromaDB** vector store
- **sentence-transformers** embeddings
- **Swappable LLM backend** local Ollama or remote OpenRouter via an OpenAI-compatible client

2. Architecture

The system is a three-stage pipeline, orchestrated behind two FastAPI endpoints (
The system is a three-stage pipeline, orchestrated behind two FastAPI endpoints (*POST /ingest, POST /ask*), with *GET /analytics* exposing usage metrics to the Streamlit frontend.

Ingestion

Handled in rag/ingestor.py. PyMuPDF extracts text page-by-page, then RecursiveCharacterTextSplitter chunks each page's text independently (chunk_size=700, chunk_overlap=150, separator priority ["\n\n", "\n", ". ", " "]). Chunks never cross page boundaries, preserving the document's clause structure but creating the edge case discussed in Section 6.

Retrieval

Handled in rag/retriever.py. Chunks are embedded with sentence-transformers/all-MiniLM-L6-v2 and stored in a persistent ChromaDB collection (cosine similarity). At query time, the question is embedded and the top-k most similar chunks are returned with a similarity score per chunk.

Generation

Handled in rag/generator.py. Retrieved chunks are passed to the LLM (temperature=0.1) with a system prompt that restricts it to the supplied context and requires it to declare which chunks it used via a SOURCES_USED: 1,3 line, which is parsed and cross-checked against the retrieved set. Every /ask call is logged to SQLite regardless of outcome, and the Streamlit app's Analytics tab surfaces usage patterns back to the operator.

3. Key Design Decisions

Page-Aware Chunking

Splitting per page rather than across the whole document avoids stitching unrelated clauses together, at the cost of occasionally cutting an answer in half at a page break (see Section 6).

Embedding Model: all-MiniLM-L6-v2

A small (~100M parameter), fast, free embedding model sufficient for a single-document corpus of roughly 100–150 chunks, where a larger embedder would add latency without a meaningful retrieval-quality gain at this scale.

Vector Store: ChromaDB

Chosen over a raw FAISS index for built-in persistence and metadata storage (page number per chunk) without separate index-file bookkeeping; cosine similarity is used as the distance metric.

top_k = 5 in Production (vs. top_k = 8 in Evaluation)

The retrieval-only grid search (Section 4) shows recall and MRR keep improving up to top_k=8. In practice, feeding the local LLM 8 chunks of a dense legal document increased hedging and added generation latency, for a marginal retrieval gain (top_k=6 already captures most of top_k=8's benefit). top_k=5 was kept as the shipped default as the better quality/latency trade-off for generation, not retrieval alone.

Three-Layer Hallucination Prevention

- **Pre-LLM confidence gate** if the best retrieved chunk's similarity score is below RETRIEVAL_CONFIDENCE_THRESHOLD (0.6), the system returns a fixed 'not found' response without calling the LLM at all.
- **Grounding system prompt** instructs the model to answer only from the supplied chunks and to refuse otherwise.
- **Citation verification** the model must emit SOURCES_USED: 1,3 (or none); this is parsed and checked against the chunks actually retrieved, giving an explicit, checkable signal rather than trusting prose alone.

Swappable LLM Provider

Generation goes through an OpenAI-compatible client with a configurable base_url, so the same code path serves either local Ollama (http://localhost:11434/v1/) or OpenRouter, controlled purely through environment variables.

4. Evaluation & Tuning Methodology

Retrieval quality was tuned with a grid search (notebooks/topk_chunk_overlap_tuning.ipynb) over:

- **chunk_size = {100, 500, 700}**
- **chunk_overlap = {50, 100, 150}**
- **top_k = {4, 6, 8}**

The grid search ran against 24 hand-labelled queries (each with ground-truth relevant_pages, including intentional 'trap' questions with no answer in the document). Metrics used:

- **Recall@k** : fraction of relevant pages retrieved
- **MRR**: mean reciprocal rank of the first relevant hit
- **Composite score** = 0.5 × recall + 0.5 × MRR

chunk_size	overlap	top_k	Recall@k	MRR	Composite
700	100	8	0.896	0.808	**0.852**
500	100	8	0.896	0.797	0.846
700	150	8	0.875	0.802	0.839

This grid search evaluates retrieval quality only. The decision to ship top_k=5 (and overlap=150) instead of the top-scoring **top_k=8/overlap=100** configuration was a separate, generation-side judgment call made after observing the local LLM's behaviour on real outputs the evaluation informed the trade-off rather than dictating the final config.

5. Logging & Analytics

Every call to /ask is logged to a SQLite query_logs table (fields: query, answer, found, top_score, latency_ms, created_at), independent of whether an answer was found.
Three aggregate queries are exposed via GET /analytics and rendered in the Streamlit Analytics tab:

- Most frequent questions
- No-answer queries
- Average latency

The no-answer query log is what surfaced the chunk-boundary fragmentation issue described in Section 6 turning a latent retrieval gap into something observable and debuggable, rather than running the system blind.

6. Limitations & Future Work

Known Limitations

- **Page-boundary fragmentation:** an answer whose supporting text spans a page break can be marked 'not found' even when the retrieval score clears the 0.6 threshold, because the two halves never end up in the same chunk.
- **Single-document scope** : one ChromaDB collection per deployment; re-ingestion deletes and rebuilds the whole collection, with no multi-document or incremental-update support.
- **No query expansion or reranking** : retrieval is a single dense-embedding pass with no HyDE-style query rewriting and no cross-encoder reranker on top of the initial similarity ranking.
- **Citation parsing brittleness:** correctness depends on the LLM reliably emitting the SOURCES_USED: 1,3 format; if it doesn't, the system falls back to the highest-scoring retrieved chunk.

Future Work

- Overlap chunks across page boundaries (or a sliding window) to close the fragmentation gap.
- Add a document_id metadata field to support multiple source documents.
- Add a lightweight reranker on top of the dense retrieval pass.
- Feed the no-answer analytics signal into periodic re-tuning of chunk/top_k parameters.

7. Conclusion

The system pairs a deliberately tuned retrieval pipeline with multi-layer grounding to minimise hallucinated answers, and closes the loop with SQLite-backed usage logging that makes retrieval gaps observable rather than silent.

Trade-offs most notably top_k and chunk overlap were measured via an explicit grid search and then adjusted based on real generation behaviour. The known remaining gaps (page-boundary fragmentation, single-document scope, no reranking) are identified with concrete next steps for future development.